

Desenvolvimentos matemáticos e numéricos em escoamentos bifásicos aplicados a processos de refino

Documentação

*Adenilso Silva Simão
Antônio Castelo Filho
Fabricio Simeoni de Sousa
Leandro Franco de Souza*

Sumário

1	Documentação científica	2
1.1	Malhas hierárquicas	2
1.2	Diferenças finitas generalizadas	2
1.3	Interpolações	4
1.3.1	Aproximação polinomial	4
1.3.2	Aproximação pelo Método dos Mínimos Quadrados	5
1.3.3	Cálculo dos coeficientes	6
1.3.4	Interpolações por MMQ com peso	7
1.4	Propriedades variáveis	7
1.5	Distribuição de variáveis	7
1.5.1	Variáveis no centro da célula	7
1.5.2	Variáveis nas facetas da célula	9
1.6	Cálculo de derivadas espaciais	9
1.6.1	Derivada primeira de p na direção x_i	9
1.6.2	Derivada primeira de u_i na direção x_i	9
1.6.3	Derivada primeira de u_i na direção x_j	9
1.6.4	Derivada segunda de u_i na direção x_i	13
1.7	Resolução da equação de Poisson	13
1.8	Simulação de escoamento incompressível 2D	16
1.8.1	Método da projeção de Chorin	16
1.8.2	Método da solução manufaturada	16
2	Documentação do usuário	18
2.1	Tutorial 1 – Gerando uma malha 2D	18
2.2	Tutorial 2 – Refinamento de células	19
2.2.1	Selecionando células através de janelas – 2D	20
2.3	Tutorial 3 - DIM = 3	20
2.4	Tutorial 4 - Usando iteradores	21
2.5	Tutorial 5 - Domínios e condições de contorno	24
2.6	Tutorial 6 - Poisson 2D	27
2.7	Tutorial 7 - Equação de Helmholtz vetorial nas faces	28
2.8	Tutorial 8 - Poisson paralelo 2D	29
2.9	Tutorial 9 – Equação de Advecção e Métodos Numéricos	30
2.10	Arquivos .amr	31
3	Documentação do programador	33
	Referências	33

Capítulo 1

Documentação científica

Este capítulo descreve os métodos utilizados para resolução numérica de equações diferenciais parciais (em especial as equações de Navier-Stokes) em uma malha hierárquica do tipo “HiG-tree”. São descritos detalhes sobre este tipo de malha, bem como os métodos utilizados para discretização por diferenças finitas generalizadas e interpolações necessárias entre os diferentes níveis de refinamento. Finalmente são apresentados os mecanismos necessários para a resolução de uma equação de Poisson em um domínio genérico.

1.1 Malhas hierárquicas

O desenvolvimento dos métodos apresentados neste documento é baseado em uma malha cartesiana do tipo hierárquica, contendo elementos de tamanhos diferentes. De agora em diante, tais malhas serão denotadas como `HiG` (do inglês, “*Hierarchical Grid*”) e a estrutura de dados utilizada para representá-la será denominada `HiG-Tree`. Tal estrutura é baseada em subdivisões espaciais recursivas arbitrárias, e pode ser representada por um árvore m -tree. Um exemplo clássico deste tipo de malha são quadtree em 2D e octree em 3D.

A malha `HiG` é baseada em uma matriz regular de elementos de mesmo tamanho, e, a partir destes, subdivisões arbitrárias podem ser definidas de forma recursiva, obtendo-se uma malha irregular (não-estruturada). Um exemplo desta malha e sua respectiva estrutura de árvore podem ser vistos na figura 1.1. Apesar de o exemplo da figura 1.1 ser ilustrado em duas dimensões, esta representação é genérica o suficiente para ser utilizada em qualquer número de dimensões.

1.2 Diferenças finitas generalizadas

O método adotado para discretização de equações diferenciais é o método de diferenças finitas. Tal método é baseado na expansão em série de Taylor da função a ser aproximada e de suas derivadas, cuja omissão de termos de alta ordem desta série resulta em fórmulas adequadas para as aproximações.

Um exemplo clássico é a discretização de uma equação de Poisson do tipo

$$\nabla^2 u = f,$$

com condições de contorno adequadas. Simplificando o domínio por um retângulo em um espaço de dimensão dois $\Omega = [a, b] \times [c, d]$, e utilizando uma malha cartesiana regular (do tipo grid) com espaçamentos δx e δy , obtém-se a clássica fórmula do laplaciano de 5 pontos, dada por

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\delta x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\delta y^2} + \mathcal{O}(\delta x^2 + \delta y^2) = f_{i,j},$$

ou ainda, desprezando-se os termos de ordem 2,

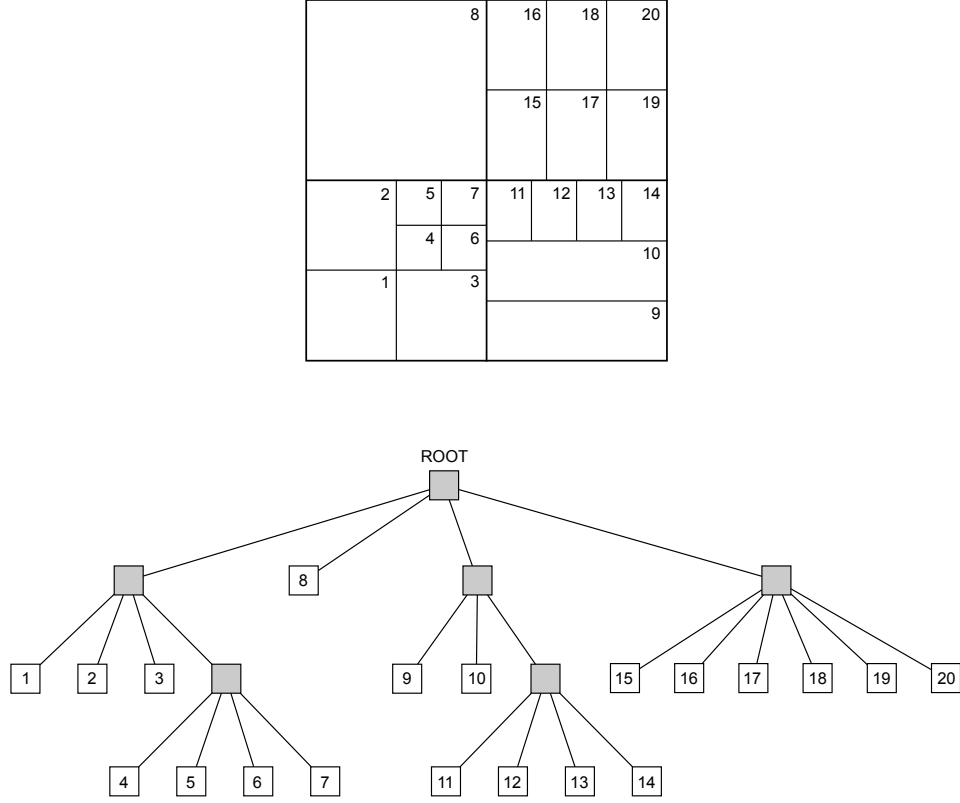


Figura 1.1: Exemplo de uma malha tipo HiG e sua representação na forma de árvore (HiG-Tree).

$$\frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{\delta x^2} + \frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{\delta y^2} = f_{i,j},$$

onde $U_{i,j}$ representa uma aproximação para a função $u_{i,j}$, avaliada em $(x_i, y_j) = (a + i\delta x, b + j\delta y)$, e $f_{i,j} = f(x_i, y_j)$. Com esta fórmula, mais as condições de contorno, é possível montar um sistema da forma

$$L\mathbf{u} = \mathbf{f},$$

cujas soluções são o vetor de aproximações $\mathbf{u} = (U_{1,1}, U_{1,2}, \dots)$ para a função u original, avaliada nos vértices da malha cartesiana.

Como visto na seção anterior, o objetivo é utilizar diferenças finitas em uma malha cartesiana irregular, então tal aproximação deve ser modificada, pois uma discretização por um laplaciano de 5 pontos resultaria no envolvimento de valores não conhecidos no interior da malha. Desta forma é preciso realizar um procedimento de interpolação que seja genérico o suficiente para englobar qualquer configuração de células (como no exemplo da figura 1.1), e que seja preciso o suficiente para não deteriorar a ordem de convergência do método numérico.

Uma clara desvantagem da utilização de malhas irregulares como as exemplificadas anteriormente é que uma numeração do tipo matricial (i, j) não é mais possível, sendo necessária a definição de uma enumeração dos nós válidos da malha. Tal enumeração é fornecida pela estrutura `HiG-Tree`, contudo, diferentes formas de enumeração podem ser implementadas, com impacto direto na estrutura da matriz do sistema resultante, e consequentemente no desempenho dos métodos numéricos utilizados para resolvê-lo.

Considere o caso ilustrado na figura 1.2, onde um estêncil de cinco pontos é utilizado para discretizar as derivadas no ponto x_C . É fácil notar que, devido a configuração irregular da malha, não existe nó da

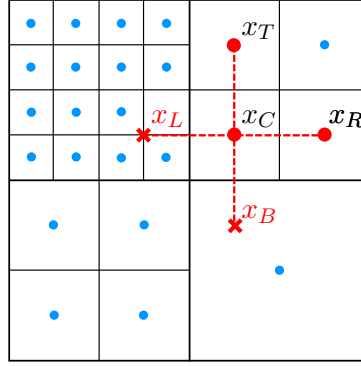


Figura 1.2: Exemplo de um estêncil resultando em pontos a serem interpolados (x_L e x_B).

malha que armazene um valor de u para os pontos x_L e x_B , ou seja, não existem incógnitas U_L e U_B para o sistema linear. Neste exemplo, é necessário obter então uma interpolação dos nós vizinhos, pertencentes à malha, mais próximos às incógnitas não existentes. Digamos que uma interpolação adequada resulte na seguinte expressão para U_L

$$U_L \approx w_1 U_{j_1} + w_2 U_{j_2} + \dots + w_n U_{j_n},$$

sendo $U_{j_1}, U_{j_2}, \dots, U_{j_n}$ valores conhecidos na malha. Escrevendo a fórmula usual do laplaciano de 5 pontos para o ponto x_C , obtém-se uma equação para U_C , a saber,

$$\frac{U_L - 2U_C + U_R}{\delta x^2} + \frac{U_T - 2U_C + U_B}{\delta y^2} = f_C,$$

que pode ainda ser reescrita como

$$a_L U_L + a_R U_R + a_T U_T + a_B U_B + a_C U_C = f_C,$$

onde

$$a_L = \frac{1}{\delta x^2}, \quad a_R = \frac{1}{\delta x^2}, \quad a_T = \frac{1}{\delta y^2}, \quad a_B = \frac{1}{\delta y^2}, \quad a_C = -\frac{2}{\delta x^2} - \frac{2}{\delta y^2}.$$

Substituindo o valor interpolado de U_L na equação, obtém-se

$$a_L (w_1 U_{j_1} + w_2 U_{j_2} + \dots + w_n U_{j_n}) + a_R U_R + a_T U_T + a_B U_B + a_C U_C = f_C,$$

gerando-se novos coeficientes e uma equação. Note que o mesmo deve ser feito para U_B , para que a equação envolva finalmente somente valores conhecidos da malha. Note que este processo pode ser repetido arbitrariamente para qualquer valor que ainda não esteja definido na malha, e que incógnitas repetidas podem ser agrupadas em um único coeficiente multiplicativo. Desta forma, obtém-se uma expressão que é uma combinação linear formada somente por incógnitas verdadeiras, para valores definidos na malha, que define a equação para a incógnita U_C no sistema linear.

A próxima seção explica como obter uma interpolação que seja suficientemente genérica e precisa, e como obter os pesos $w_{j_1}, w_{j_2}, \dots, w_{j_n}$ necessários.

1.3 Interpolações

1.3.1 Aproximação polinomial

Um dos objetivos do método descrito aqui é obter uma aproximação para o valor de uma propriedade em um dado ponto (x, y, z) em função dos valores da propriedade em m outros pontos $(x_1, y_1, z_1), \dots, (x_m, y_m, z_m)$.

Os valores nos m pontos não são conhecidos a priori. Denotamos o valor em um ponto (x, y, z) por $P(x, y, z)$ um polinômio de grau no máximo r ,

$$P(x, y, z) = \alpha_{i_1, j_1, k_1} x^{i_1} y^{j_1} z^{k_1} + \dots + \alpha_{i_n, j_n, k_n} x^{i_n} y^{j_n} z^{k_n} = c^t \alpha,$$

onde $0 \leq i_s + j_s + k_s \leq r$, para $s = 1, \dots, n$. Observe que poderia ser uma combinação de funções não polinomiais.

Assim, o método deve obter os coeficientes w_i , para $1 \leq i \leq m$, de forma que

$$P(x, y, z) = \sum_{i=1}^m w_i P(x_i, y_i, z_i) = w^t b.$$

Assim,

$$\begin{aligned} b = \begin{pmatrix} P(x_1, y_1, z_1) \\ \vdots \\ P(x_m, y_m, z_m) \end{pmatrix} &= \begin{pmatrix} x_1^{i_1} y_1^{j_1} z_1^{k_1} & \dots & x_1^{i_n} y_1^{j_n} z_1^{k_n} \\ \vdots & & \vdots \\ x_m^{i_1} y_m^{j_1} z_m^{k_1} & \dots & x_m^{i_n} y_m^{j_n} z_m^{k_n} \end{pmatrix} \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{pmatrix} = \\ &= \begin{pmatrix} a_1^t \\ \vdots \\ a_m^t \end{pmatrix} \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{pmatrix} = A \alpha \end{aligned}$$

Logo,

$$w^t b = w^t A \alpha = c^t \alpha$$

e uma solução é dada por $w^t A = c^t$, ou

$$A^t w = c.$$

Se $m \geq n$ e A tem posto máximo n , w pode ser obtido pelo método dos mínimos quadrados.

1.3.2 Aproximação pelo Método dos Mínimos Quadrados

Fazendo a decomposição QR de A , temos:

$$\begin{aligned} A_{m \times n} &= \begin{pmatrix} A_{1n \times n} \\ A_{2m-n \times n} \end{pmatrix} = Q_{m \times m} R_{m \times n}^* \\ &= Q_{m \times m} \begin{pmatrix} R_{n \times n} \\ 0_{m-n \times n} \end{pmatrix} = \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \end{pmatrix} \begin{pmatrix} R \\ 0 \end{pmatrix} \end{aligned}$$

Assim,

$$\begin{aligned} w &= A^+ c^t = A (A^t A)^{-1} c^t = QR^* (R^{*t} Q^t Q R^*)^{-1} c^t = QR^* (R^t R)^{-1} c^t = \\ &= QR^* R^{-1} (R^t)^{-1} c^t = Q \begin{pmatrix} R \\ 0 \end{pmatrix} R^{-1} (R^t)^{-1} c^t = Q \begin{pmatrix} (R^t)^{-1} \\ 0 \end{pmatrix} c^t \end{aligned}$$

Fazendo $u = Q^t w$ e $u = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}$, temos:

$$u = Q^t w = \begin{pmatrix} (R^t)^{-1} \\ 0 \end{pmatrix} c^t$$

$$u = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} (R^t)^{-1} \\ 0 \end{pmatrix} c^t$$

$$\begin{cases} u_1 = (R^t)^{-1} c^t \\ u_2 = 0 \end{cases}$$

Daí, resolvendo o sistema triangular inferior $R^t u_1 = c^t$, temos $w = Qu$, com $u_2 = 0$.

Resumindo, são dados A e c^t e queremos w , então são feitos três passos:

1. Fazer a decomposição QR de A , isto é, $A = Q R^*$;
2. Resolver o sistema triangular inferior $R^t u_1 = c^t$;
3. Fazer $w = Qu$ com $u_2 = 0$.

Observe que para o caso de $m = n$, a aproximação polinomial P será o polinômio interpolador.

1.3.3 Cálculo dos coeficientes

Seja

$$C = \begin{pmatrix} A_1 & B_1 \\ A_2 & B_2 \end{pmatrix} = \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \end{pmatrix} \begin{pmatrix} R & 0 \\ 0 & I \end{pmatrix} = \begin{pmatrix} Q_{11}R & Q_{12} \\ Q_{21}R & Q_{22} \end{pmatrix}.$$

Desde que,

$$I_m = Q^t Q = \begin{pmatrix} Q_{11}^t & Q_{21}^t \\ Q_{12}^t & Q_{22}^t \end{pmatrix} \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \end{pmatrix} =$$

$$= \begin{pmatrix} Q_{11}^t Q_{11} + Q_{21}^t Q_{21} & Q_{11}^t Q_{12} + Q_{21}^t Q_{22} \\ Q_{12}^t Q_{11} + Q_{22}^t Q_{21} & Q_{12}^t Q_{12} + Q_{22}^t Q_{22} \end{pmatrix} = \begin{pmatrix} I_n & 0 \\ 0 & I_{m-n} \end{pmatrix},$$

Temos que,

$$A^t B = \begin{pmatrix} A_1^t & A_2^t \end{pmatrix} \begin{pmatrix} B_1 \\ B_2 \end{pmatrix} = A_1^t B_1 + A_2^t B_2 = R (Q_{11}^t Q_{12} + Q_{21}^t Q_{22}) = 0,$$

$$B^t B = \begin{pmatrix} B_1^t & B_2^t \end{pmatrix} \begin{pmatrix} B_1 \\ B_2 \end{pmatrix} = B_1^t B_1 + B_2^t B_2 = Q_{12}^t Q_{12} + Q_{22}^t Q_{22} = I_{m-n},$$

e

$$C^t C = \begin{pmatrix} A_1^t & A_2^t \\ B_1^t & B_2^t \end{pmatrix} \begin{pmatrix} A_1 & B_1 \\ A_2 & B_2 \end{pmatrix} = \begin{pmatrix} R^t R & 0 \\ 0 & I \end{pmatrix}.$$

Logo $\text{Det}(C) = \pm \text{Det}(R) = R_{1,1} \cdots R_{n,n}$, e se A tem posto máximo $R_{i,i} \neq 0$ e

$$C^t w = \begin{pmatrix} A_1^t & A_2^t \\ B_1^t & B_2^t \end{pmatrix} w = \begin{pmatrix} c \\ 0 \end{pmatrix},$$

tem solução única.

Se $m = n$, $C^t = A_1^t = (a_1 \cdots a_n)$. Logo se $x = x_i (i = 1, \dots, n)$ tem-se $c = a_i$ e $C^t w = d$, segue que $A_1 w = c$ que implica que

$$(a_1 \cdots a_i \cdots a_n) w = a_i,$$

tem solução $w_k = 1$ se $i = k$ e $w_k = 0$ se $i \neq k$.

Se $m > n$ e $x = x_i (i = 1, \dots, n)$,

$$C^t w = \begin{pmatrix} A_1^t & A_2^t \\ B_1^t & B_2^t \end{pmatrix} w = \begin{pmatrix} a_1 & \cdots & a_i & \cdots & a_m \\ b_1 & \cdots & b_i & \cdots & b_m \end{pmatrix} w = \begin{pmatrix} c \\ 0 \end{pmatrix} = \begin{pmatrix} a_i \\ 0 \end{pmatrix}.$$

Como

$$C^t = \begin{pmatrix} A_1^t & A_2^t \\ B_1^t & B_2^t \end{pmatrix} = \begin{pmatrix} R^t & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} Q_{11}^t & Q_{21}^t \\ Q_{12}^t & Q_{22}^t \end{pmatrix} = \begin{pmatrix} R^t & 0 \\ 0 & I \end{pmatrix} Q^t,$$

$$\begin{pmatrix} A_1^t & A_2^t \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} R^t & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} Q_{11}^t & Q_{21}^t \\ Q_{12}^t & Q_{22}^t \end{pmatrix} = \begin{pmatrix} R^t & 0 \\ 0 & 0 \end{pmatrix} Q^t,$$

que implica que

$$\begin{pmatrix} a_i \\ 0 \end{pmatrix} = \begin{pmatrix} A_1^t & A_2^t \\ 0 & 0 \end{pmatrix} e_i = \begin{pmatrix} R^t & 0 \\ 0 & 0 \end{pmatrix} Q^t e_i.$$

Logo,

$$C^t w = d$$

$$\begin{pmatrix} R^t & 0 \\ 0 & I \end{pmatrix} Q^t w = \begin{pmatrix} R^t & 0 \\ 0 & 0 \end{pmatrix} Q^t e_i.$$

$$w = Q \begin{pmatrix} R^{-t} & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} R^t & 0 \\ 0 & 0 \end{pmatrix} Q^t e_i.$$

$$w = Q \begin{pmatrix} I & 0 \\ 0 & 0 \end{pmatrix} Q^t e_i.$$

1.3.4 Interpolações por MMQ com peso

Complementar com MMQ com peso.

1.4 Propriedades variáveis

O que muda para ρ variável.

1.5 Distribuição de variáveis

Com o objetivo de evitar o aparecimento de oscilações não físicas nas simulações numéricas, foram desenvolvidas técnicas de distribuição de variáveis de forma deslocada, ou seja, pode-se atribuir variáveis no centro da célula, ou nas facetas da mesma. O esquema mais adotado pelos pesquisadores da área de simulação de escoamento de fluidos, quando se adota formulação com variáveis primitivas, em escoamentos incompressíveis, é a distribuição da pressão nos centros das células, e das velocidades nas facetas da mesma, conforme pode ser visto na figura 1.3.

1.5.1 Variáveis no centro da célula

A pressão, densidade, entalpia total, etc, são normalmente distribuídas nos centros das células. A figura 1.4 ilustra a distribuição da pressão em uma malha com 3 níveis de refinamento.

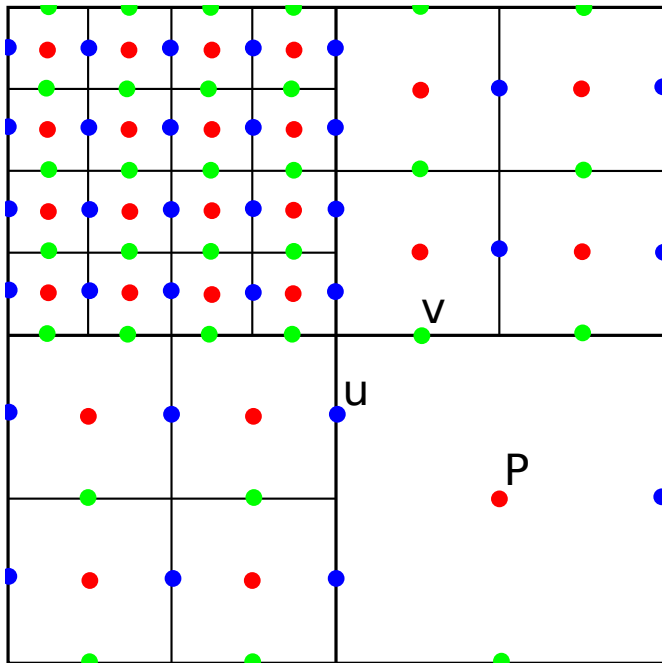


Figura 1.3: Malha deslocada.

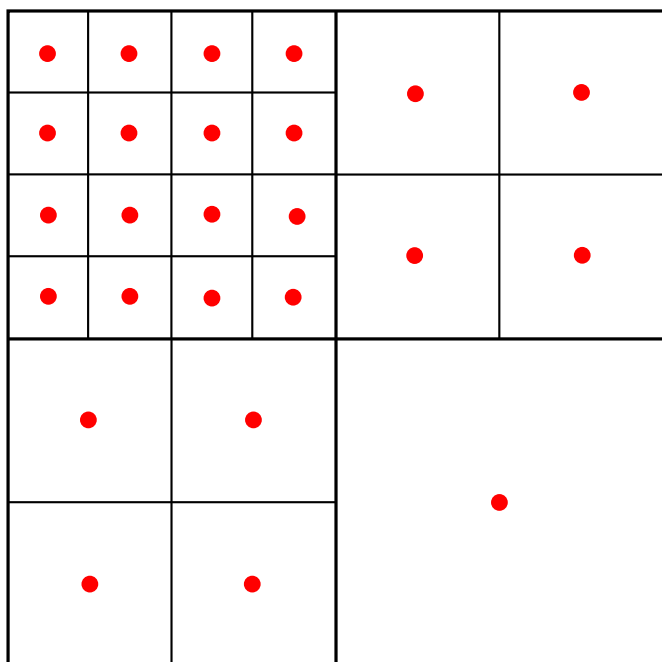


Figura 1.4: Distribuição da Pressão no centro das células.

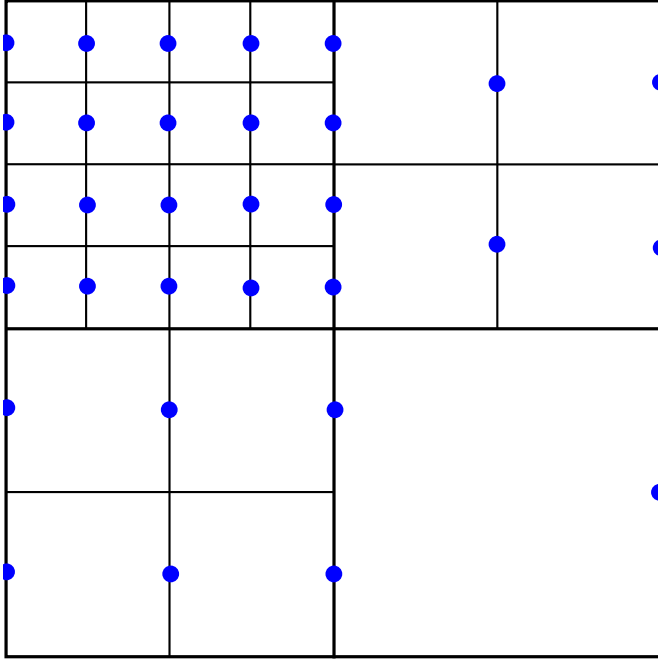


Figura 1.5: Componente da velocidade u_1 nas facetas da células.

1.5.2 Variáveis nas facetas da célula

As variáveis provenientes da decomposição da velocidade em vetores (u_i) são distribuídas nas facetas das células. A distribuição da componente da velocidade u_1 em uma malha com 3 níveis de refinamento é apresentada na figura 1.5 e a componente da velocidade u_2 nesta mesma malha é apresentada na figura 1.6.

1.6 Cálculo de derivadas espaciais

Com a utilização da malha deslocada, temos que o cálculo das derivadas espaciais de cada variável distribuída em uma localização diferente tem uma forma diferente de cálculo. Abaixo são apresentadas as variações possíveis numa simulação de escoamento.

1.6.1 Derivada primeira de p na direção x_i

Neste caso a variável é definida no centro da célula e pretende-se calcular a derivada espacial na direção x_i , na faceta da célula. A figura 1.7 ilustra as possibilidades de cálculo da derivada na posição marcada com um x de uma variável distribuída no centro da célula na direção x_1 . A forma equivalente para o cálculo da derivada na direção x_2 , é apresentada na figura 1.8.

1.6.2 Derivada primeira de u_i na direção x_i

Nas equações de Navier-Stokes faz-se necessário o cálculo das primeiras derivadas de variáveis distribuídas nas facetas das células. A figura 1.9 ilustra o cálculo da derivada na posição marcada com um x da componente u_1 na direção x_1 . A malha adotada tem 3 níveis de refinamento para ilustrar as diversas possibilidades neste cálculo. O cálculo da componente u_2 na direção x_2 é apresentado na figura 1.10.

1.6.3 Derivada primeira de u_i na direção x_j

A figura 1.11 ilustra o cálculo da derivada na posição marcada com um x da componente u_1 na direção x_2 . O cálculo da componente u_2 na direção x_1 é apresentado na figura 1.12.

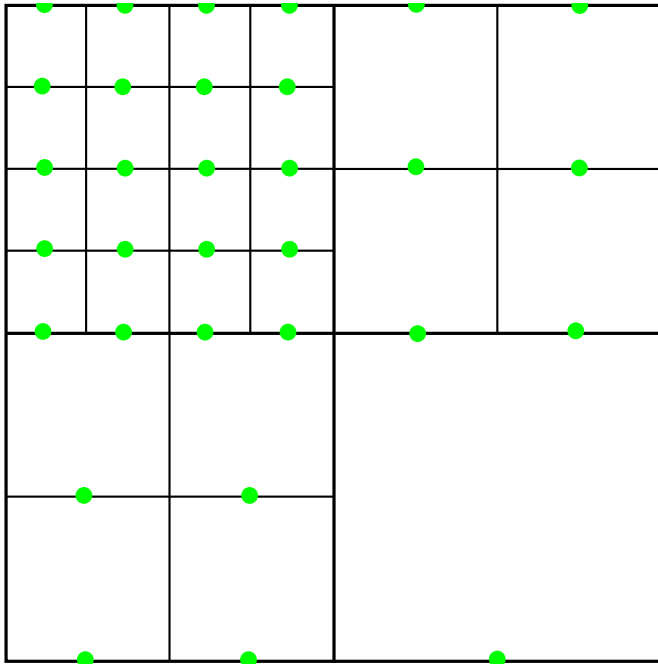


Figura 1.6: Componente da velocidade u_2 nas facetas da células.

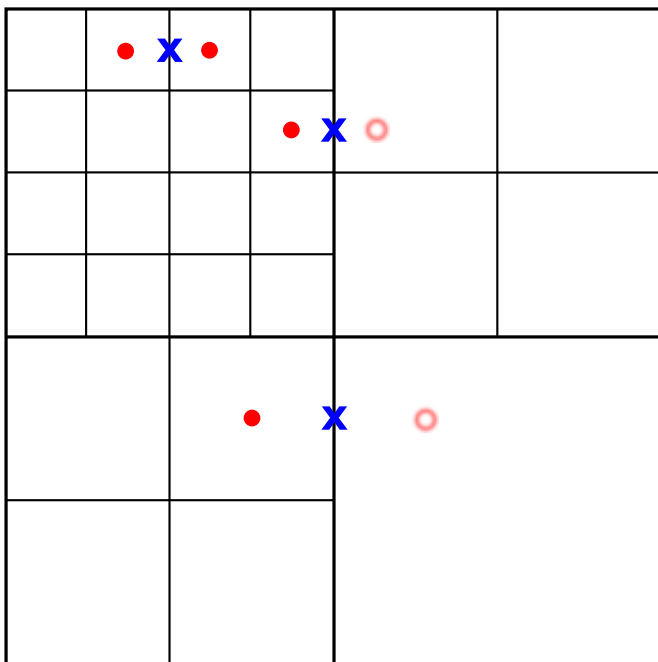


Figura 1.7: Derivada de p na direção x_1 .

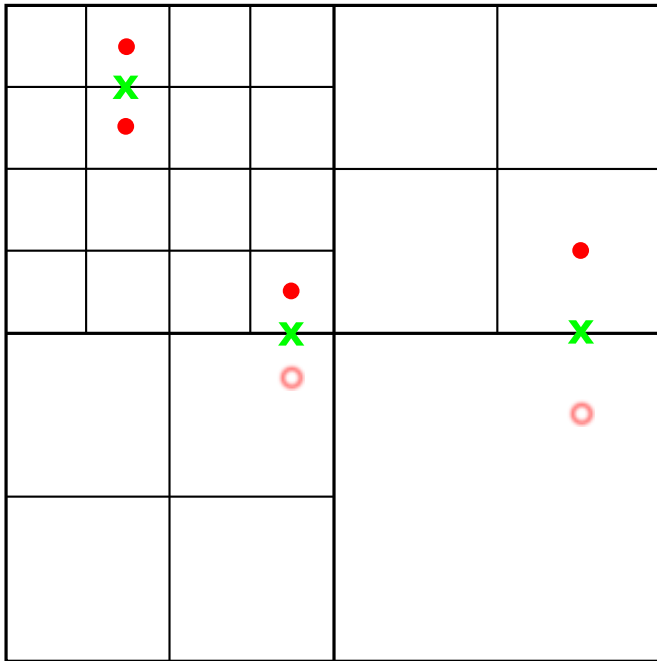


Figura 1.8: Derivada de p na direção x_2 .

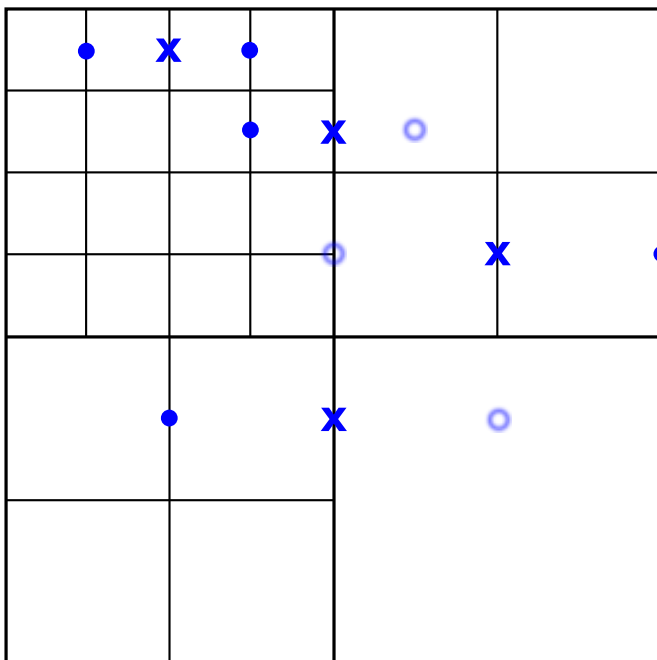


Figura 1.9: Derivada de u_1 na direção x_1 .

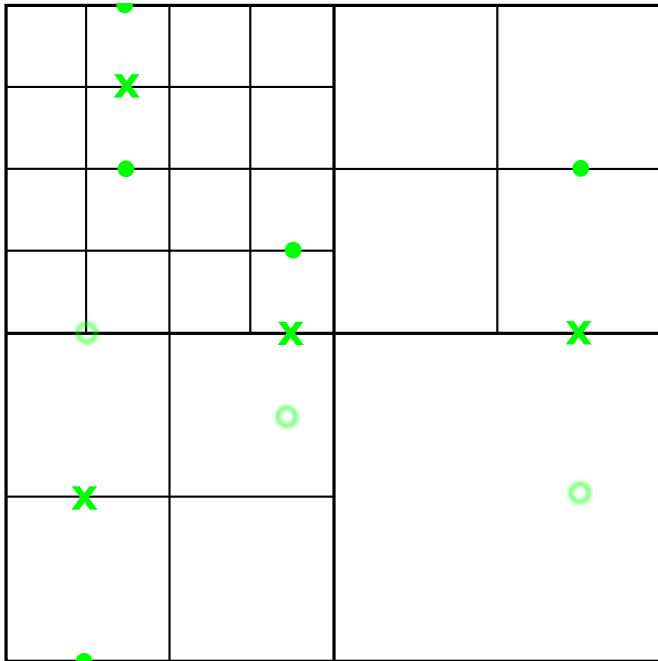


Figura 1.10: Derivada de u_2 na direção x_2 .

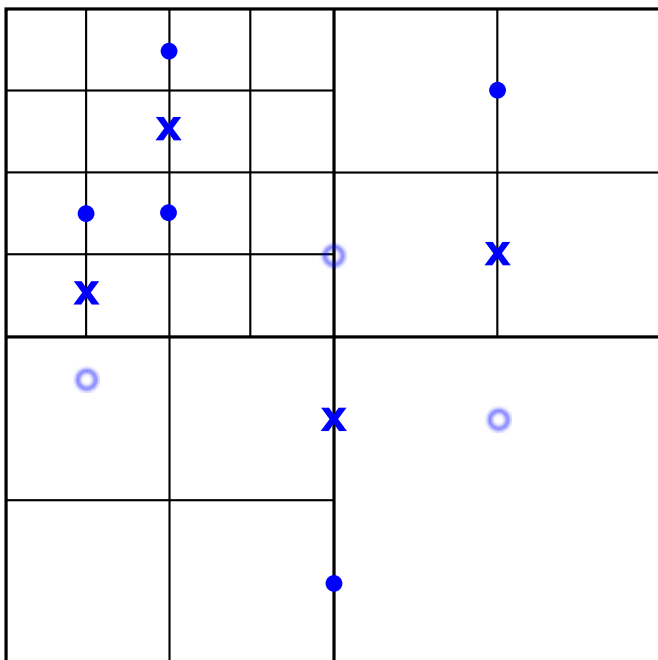


Figura 1.11: Derivada de u_1 na direção x_2 .

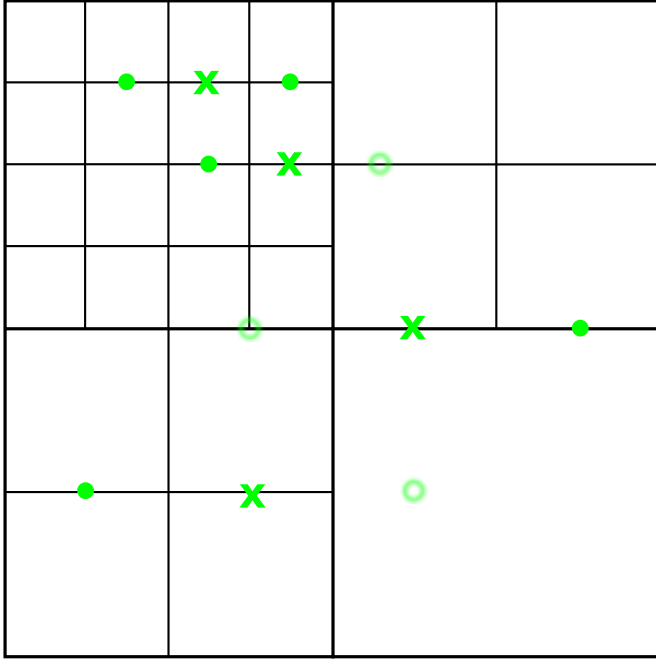


Figura 1.12: Derivada de u_2 na direção x_1 .

1.6.4 Derivada segunda de u_i na direção x_i

O cálculo da segunda derivada de variáveis definidas nas facetas das células são apresentadas nas figuras 1.13 a 1.16. A posição onde a derivada é calculada é marcada com um x . As diversas configurações dentro da malha com 3 níveis de refinamento são ilustradas.

1.7 Resolução da equação de Poisson

Dada uma variável $p(x, y)$ distribuída no centro das células e um campo de velocidade dado por $u^*(x, y)$, distribuído nas facetas das células, pretende-se resolver a equação de Poisson abaixo:

$$\nabla^2 p = \nabla \cdot u^*$$

Para analisar a convergência do método propõe-se um Método de Solução Manufaturada (MMS - do inglês *Method of Manufactured Solution*) onde as componentes da velocidade são dadas por:

$$u_1(x, y) = -\sin(x)\cos(y)$$

e

$$u_2(x, y) = -\cos(x)\sin(y)$$

utilizando as componentes de velocidade acima descritas, em um domínio $0 \leq x, y \leq \pi$ teremos que a solução desta equação para a pressão p é dada por:

$$p(x, y) = \cos(x)\cos(y)$$

a adoção desta solução garante que a condição de contorno do tipo Neumann é aplicada em todos os contornos do domínio. Para se obter uma solução única, é fixado o valor da pressão em um ponto do contorno. Desta forma a solução desta equação é bem próxima ao que desejamos para aplicação em Navier-Stokes.

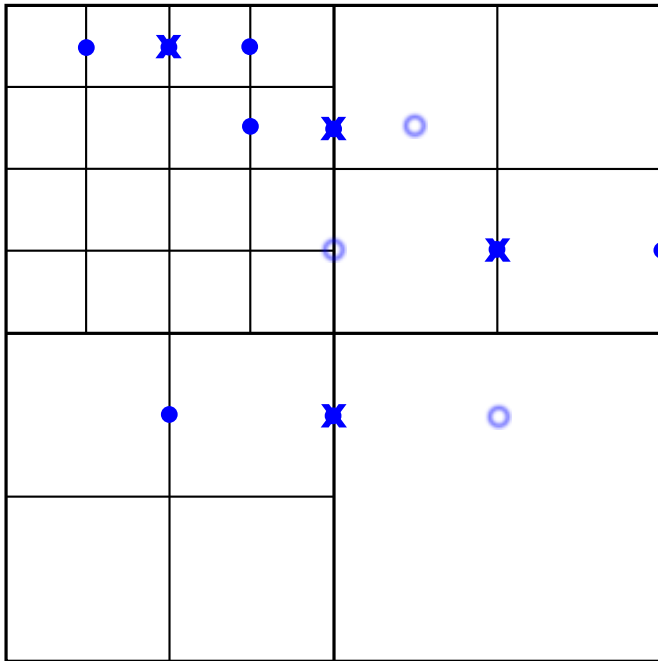


Figura 1.13: Derivada segunda de u_1 na direção x_1 .

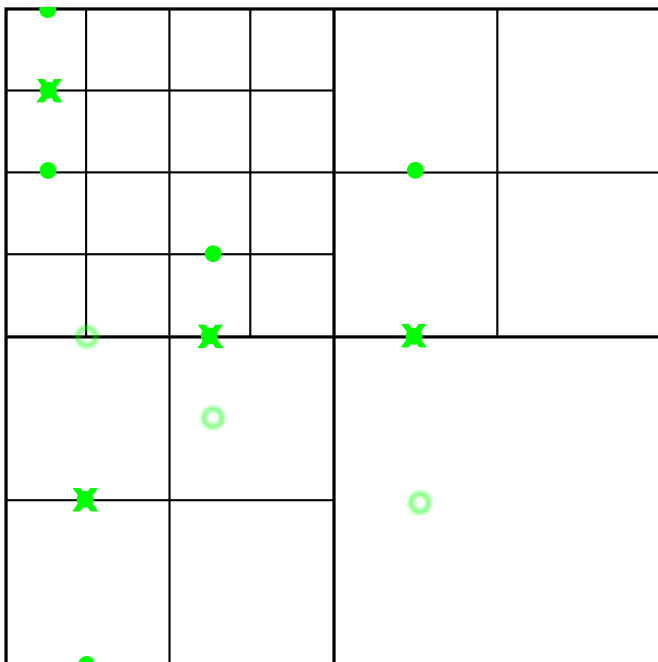


Figura 1.14: Derivada segunda de u_2 na direção x_2 .

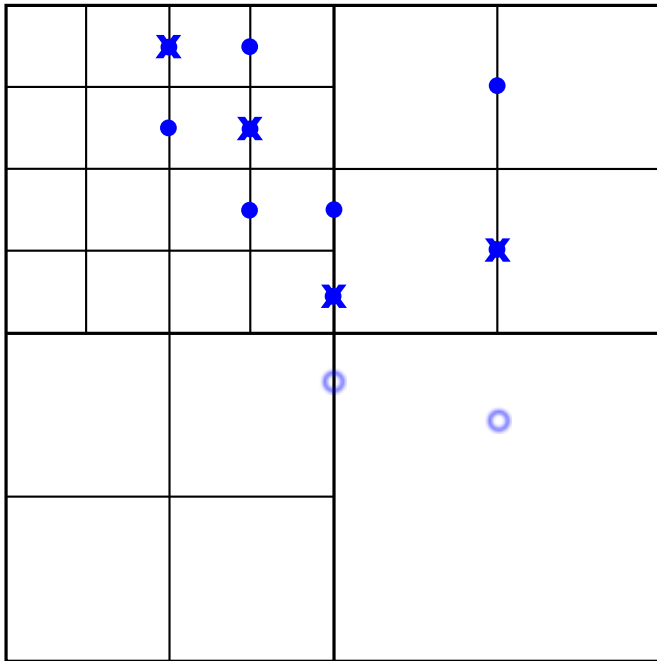


Figura 1.15: Derivada segunda de u_1 na direção x_2 .

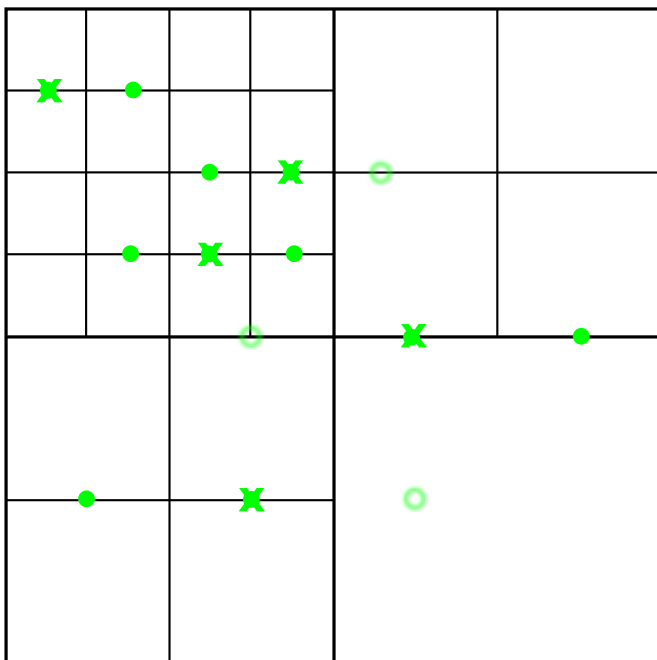


Figura 1.16: Derivada segunda de u_2 na direção x_1 .

1.8 Simulação de escoamento incompressível 2D

A dinâmica de um escoamento bidimensional incompressível de um único fluido sem transferência de calor, pode ser modelado pelas equações de Navier-Stokes e da continuidade dados abaixo:

$$\frac{\partial u_i}{\partial t} + u_j \frac{\partial u_i}{\partial x_j} = -\frac{1}{\rho} \frac{\partial p}{\partial x_i} + \nu \frac{\partial^2 u_i}{\partial x_j^2} + s_i, \quad (1.1)$$

$$\frac{\partial u_i}{\partial x_i} = 0, \quad (1.2)$$

onde ν é viscosidade cinemática, ρ é a densidade, u_i o componente da velocidade, p é pressão, t é o tempo, x_i as coordenadas espaciais e s_i o termo fonte (gravidade, coriolis, etc.).

Por razões didáticas, propõe-se inicialmente a utilização do método de Euler explícito para avançar a Eq. (1.1) no tempo. O sistema de EDPs Eqs. (1.1) e (1.2) é resolvido utilizando o método da projeção de Chorin [1] descrita abaixo.

1.8.1 Método da projeção de Chorin

Com o objetivo de se obter um campo de velocidade que respeite a continuidade, e como não há uma equação para a pressão p , as equações de Navier-Stokes (Eq. (1.1)) podem ser discretizada da seguinte forma [1]:

$$\frac{u_i^* - u_i^n}{\Delta t} = -u_j^n \frac{\partial u_i^n}{\partial x_j} + \nu \frac{\partial^2 u_i^n}{\partial x_j^2} + s_i,$$

onde o valor de u_i^* é um valor intermediário que deverá ser corrigido através da equação:

$$\frac{u_i^{n+1} - u_i^*}{\Delta t} = -\frac{1}{\rho} \frac{\partial p^{n+1}}{\partial x_i},$$

ou seja,

$$u_i^{n+1} = u_i^* - \Delta t \frac{1}{\rho} \frac{\partial p^{n+1}}{\partial x_i}.$$

Para que isto funcione, pode-se obter a equação de Poisson abaixo, que deverá ser resolvida para poder-mos avançar com a discretização temporal:

$$\frac{\partial^2 p^{n+1}}{\partial x_i^2} = \frac{\rho}{\Delta t} \frac{\partial u_i^*}{\partial x_i}$$

Utilizando-se este método obtém-se uma distribuição de velocidade e pressão que respeitam a condição de incompressibilidade. Desta forma pode-se realizar uma simulação até atingir-se o estado estacionário (quando as propriedades não variam mais com o tempo), ou até que se atinja um determinado tempo ou mesmo um estado periódico no tempo.

1.8.2 Método da solução manufaturada

Para verificação do método numérico que simula o escoamento bidimensional incompressível propõe-se resolver o problema clássico de fluxo laminar dentro de uma cavidade quadrada de dimensões $0 < x_1, x_2 < 1$, na qual a tampa se move. Para este caso, existe uma solução analítica dada por [2]. As condições de contorno para as componentes de velocidades u_i são do tipo Dirichlet, sendo nula em todos os pontos, exceto em $x_2 = 1$, onde temos:

$$u_1(x_1, 1) = 16(x_1^4 - 2x_1^3 + x_1^2).$$

Um termo fonte s_i está presente somente na equação da quantidade de movimento na direção x_2 e é dado como:

$$\begin{aligned} s_1 &= 0 \\ s_2 &= -8\nu[24F(x_1) + 2f^{(1)}(x_1)g^{(2)}(x_2) + f^{(3)}(x_1)g(x_2)] \\ &\quad -64[F_2(x_1)G_1(x_2) - g(x_2)g^{(1)}(x_2)F_1(x_1)], \end{aligned}$$

onde

$$\begin{aligned} f(x_1) &= x_1^4 - 2x_1^3 + x_1^2, \\ g(x_2) &= x_2^4 - x_2^2, \\ F(x_1) &= \int f(x_1)dx_1 = 0.2x_1^5 - 0.5x_1^4 + x_1^3/3, \\ F_1(x_1) &= f(x_1)f^{(2)}(x_1) - [f^{(1)}(x_1)]^2 = -4x_1^6 + 12x_1^5 - 14x_1^4 + 8x_1^3 - 2x_1^2, \\ F_2(x_1) &= \int f(x_1)f^{(1)}(x_1)dx_1 = 0.5[f(x_1)]^2, \\ G_1(x_2) &= g(x_2)g^{(3)}(x_2) - g^{(1)}(x_2)g^{(2)}(x_2) = -24x_2^5 + 8x_2^3 - 4x_2. \end{aligned}$$

A solução analítica é dada por

$$\begin{aligned} u(x_1, x_2) &= 8f(x_1)g^{(1)}(x_2) = 8(x_1^4 - 2x_1^3 + x_1^2)(4x_2^3 - 2x_2), \\ v(x_1, x_2) &= -8f^{(1)}(x_1)g(x_2) = -8(4x_1^3 - 6x_1^2 + 2x_1)(x_2^4 - x_2^2), \\ p(x_1, x_2) &= 8\nu[F(x_1)g^{(3)}(x_2) + f^{(1)}(x_1)g^{(1)}(x_2)] + \\ &\quad 64F_2(x_1)\{g(x_2)g^{(2)}(x_2) - [g^{(1)}(x_2)]^2\}. \end{aligned}$$

Sabendo-se a solução analítica do escoamento pode-se então verificar o correto funcionamento do mesmo. Além disto, caso sejam realizadas simulações com malhas com diferentes números de pontos, pode-se verificar a ordem de precisão do código como um todo.

Capítulo 2

Documentação do usuário

Esta parte da documentação é destinada a usuários do código e foi subdividida em tutoriais. Estes tutoriais tem como objetivo familiarizar o usuário com os comandos utilizados no código, começando por exemplos bem simples até os casos mais complexos. Cada tutorial é acompanhado de um código executável. Para compilá-los individualmente, basta usar o comando `make nome_do_tutorial DIM=dim_do_tuto` (nome_do_tutorial não inclui o `.c`). Para compilar todos de uma só vez, usa-se `make all`.

2.1 Tutorial 1 – Gerando uma malha 2D

Nesta seção iremos gerar uma malha bidimensional, em um domínio retangular, o qual damos o nome de *root*, e subdividi-lo em células. Esta será a malha raiz e a partir dela poder-se-á refiná-la nas regiões aonde o usuário achar conveniente. Iniciamos com uma malha bidimensional, portanto onde referencia-se a variável *DIM*, ela terá valor 2. O código completo se encontra no diretório *tutoriais*, com o nome de *tutorial01-2d.c*. A seguir será descrito o que é realizado em cada comando do código.

Primeiramente é necessário incluir as definições minimamente necessárias para a execução do código, as bibliotecas padrão do sistema e da HiG-Tree:

```
#include<stdio.h>
#include<stdlib.h>
#include "higtrees.h"
#include "higtrees-io.h"
```

A seguir, inicializa-se o bloco principal

```
int main(int argc, char *argv[])
```

A malha será gerada em um retângulo, desta forma é necessário declarar então os pontos inferior esquerdo *lp* e superior direito *hp*, além de atribuí-los um valor

```
Point lp, hp;
POINT_ASSIGN_SCALAR(lp, -1.0);
POINT_ASSIGN_SCALAR(hp, 1.0);
```

Veja que neste caso atribuímos $lp = (-1, -1)$ e $hp = (1, 1)$ através da macro `POINT_ASSIGN_SCALAR`, que substituí o código

```
lp[0] = -1.0;
lp[1] = -1.0;
hp[0] = 1.0;
hp[1] = 1.0;
```

Com os pontos *lp* e *hp* declarados, basta criar a malha com o comando

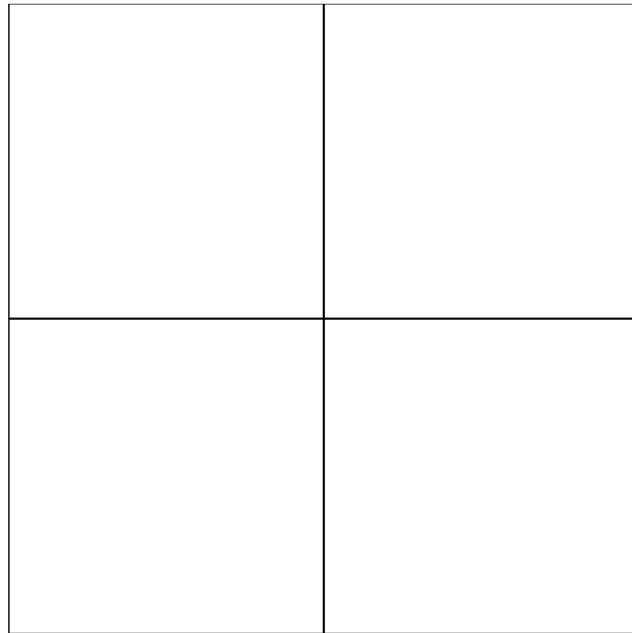


Figura 2.1: Malha gerada pelo tutorial 1.

```
hig_cell *root = hig_create_root(lp, hp);
```

Com este comando, a célula endereçada pela variável *root* foi criada. Agora é necessário subdividi-la em uma malha regular. Para este exemplo, vamos gerar uma malha de 2×2 elementos. Primeiro, declara-se o número de células desejadas em cada direção (lembrando que $DIM=2$)

```
int num_cell[DIM];  
POINT_ASSIGN_SCALAR(num_cell, 2);
```

e invocar o comando

```
hig_refine_uniform(root, num_cell);
```

para criar a malha desejada.

Para visualizar a malha, basta criar um arquivo *vtk* e imprimir a malha no arquivo. Isto pode ser realizado da seguinte forma:

```
FILE *fd = fopen("VTKS/tut01.vtk", "w");  
higio_print_in_vtk2d(fd, root);  
fclose(fd);
```

Uma imagem da malha gerada pode ser vista na figura 2.1.

Após realizados todos os cálculos a raiz tem que ser destruída para não ficar ocupando espaço em memória, o que é feito através do comando:

```
hig_destroy(root);
```

2.2 Tutorial 2 – Refinamento de células

Nesta seção iremos tomar uma malha bidimensional gerada como no exemplo anterior, mas com 4 células na direção *x* e 5 na direção *y*, e realizar o procedimento de refinamento, ou seja, tomar uma ou mais

células da malha e dividi-la em partes menores. O código a ser adotado se encontra no diretório *tutoriais*, com o nome de *tutorial02-2d.c*. A seguir será descrito o que é realizado em cada comando do código.

Após inicializada a nova malha, subdivide-se o domínio da forma descrita acima

```
int num_cell[DIM];
num_cell[0] = 4;
num_cell[1] = 5;
hig_refine_uniform(root, num_cell);
```

Digamos que seja necessário o refinamento de células próximo ao ponto $(0.5, 0.5)$. Definamos o ponto p

```
Point p;
p[0] = 0.5;
p[1] = 0.5;
```

Em seguida, selecionamos a célula a ser refinada através do comando

```
hig_cell *c = hig_get_cell_with_point(root, p);
```

Para realizar o refinamento da célula que contém o ponto p , define-se o número de células que será utilizado para este refinamento nas duas direções e em seguida é dado o comando de refinamento uniforme na célula c .

```
num_cells[0] = 2;
num_cells[1] = 2;
hig_refine_uniform(c, num_cells);
```

Como feito anteriormente, a malha pode ser impressa em arquivo *vtk* da seguinte forma

```
FILE *fd = fopen("VTKS/tut02.vtk", "w");
higio_print_in_vtk2d(fd, root);
fclose(fd);
```

A malha resultante pode ser vista na figura 2.2.

2.2.1 Selecionando células através de janelas – 2D

2.3 Tutorial 3 - DIM = 3

Nesta seção, faremos uma malha tridimensional. O procedimento é análogo ao de criar uma bidimensional, como na seção 2.2, com a única diferença que temos mais uma coordenada de liberdade para refinar. O código, do arquivo *tutorial03-2d.c*, segue da seguinte maneira:

```
Point lo, hi;
POINT_ASSIGN_SCALAR(lo, -1.0);
POINT_ASSIGN_SCALAR(hi, 1.0);
```

Criamos $lo = (-1.0, -1.0, -1.0)$ e $hi = (1.0, 1.0, 1.0)$ os pontos que definem o paralelepípedo do domínio. O macro `POINT_ASSIGN_SCALAR` coloca o mesmo valor em todas as entradas do vetor.

```
hig_cell *root = hig_create_root(lo, hi);
int nc[3] = {2, 3, 4};
hig_refine_uniform(root, nc);
```

Criamos a célula e então a refinamos com 2 células na direção x , 3 na direção y e 4 na direção z .

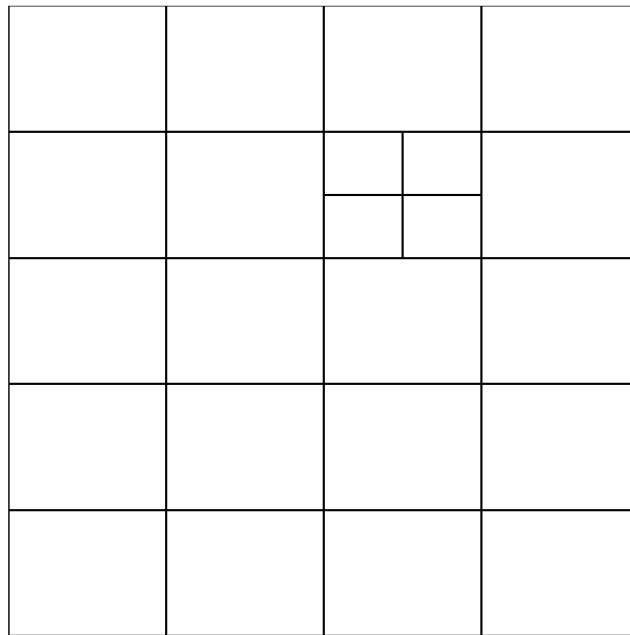


Figura 2.2: Malha gerada pelo tutorial 2.

```
Point a;
POINT_ASSIGN_SCALAR(a, 0.5);
hig_cell *c = hig_get_cell_with_point(root, a);
nc[0] = 2; nc[1] = 2; nc[2] = 2;
hig_refine_uniform(c, nc);
```

Pegamos a célula com o ponto $a = (0.5, 0.5, 0.5)$ e a refinamos com em 8 células distintas ($2 \times 2 \times 2$).

```
FILE *f = fopen("VTKS/tut03.vtk", "w");
higio_print_in_vtk3d(f, root);

fclose(f);
hig_destroy(c);
```

Por fim, como feito anteriormente, salvamos a célula criada e finalizamos o programa. A malha gerada pode ser vista na figura 2.3:

2.4 Tutorial 4 - Usando iteradores

Este código introduz o uso de iteradores para percorrer as faces ou as células de uma HiG-Tree. Existem dois tipos de iteradores: *higcit_celliterator* e *higfit_facetiterator*. Como seus nomes indicam, esses objetos iteram células e faces respectivamente. O código implementado em *tutorial04-2d.c* segue:

```
#include "higtrees-iterator.h"
```

Devemos incluir a biblioteca que contém a estrutura e os métodos dos iteradores, além das bibliotecas já mencionadas anteriormente.

```
Point lo, hi;
POINT_ASSIGN_SCALAR(lo, -1.0);
```

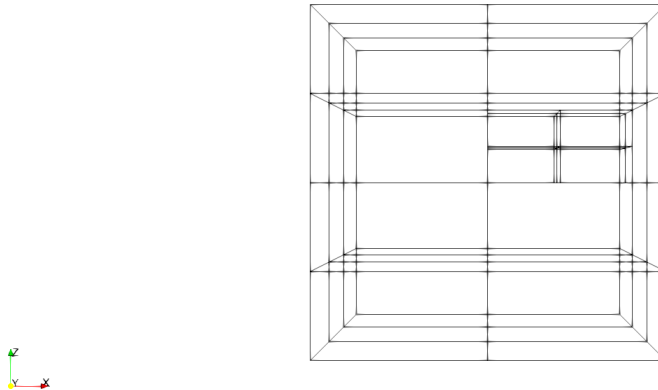


Figura 2.3: Malha gerada pelo tutorial 3

```
POINT_ASSIGN_SCALAR(hi, 1.0);

hig_cell *root = hig_create_root(lo, hi);

int nc[2] = {2,2};
hig_refine_uniform(root, nc);

Point refine_point;
POINT_ASSIGN_SCALAR(refine_point, 0.5);
hig_cell *c = hig_get_cell_with_point(root, refine_point);
hig_refine_uniform(c, nc);
FILE *fd = fopen("VTKS/tut04.vtk", "w");
higio_print_in_vtk2d(fd, root);
fclose(fd);
```

Criamos a HiG-Tree da figura 2.4. O código é análogo aos tutoriais anteriores.

Agora com a célula raiz criada, podemos iterar sobre as células filho. Existem algumas formas de criar um iterador para células, sendo a mais comum (e utilizada nesse tutorial) iterar sobre todas as folhas da árvore (todas as células atuais).

```
higcit_celliterator *cit;
printf("centers of the cells:\n");

for(cit = higcit_create_all_leaves(root);
!higcit_isfinished(cit); higcit_nextcell(cit)) {
    hig_cell *ccell = higcit_getcell(cit);
    Point center;
    hig_get_center(ccell, center);
    int id = hig_get_cid(ccell);
    printf("cell: %d \t(x= %.2f, y= %.2f)\n",
id, center[0], center[1]);
}
higcit_destroy(cit);
```

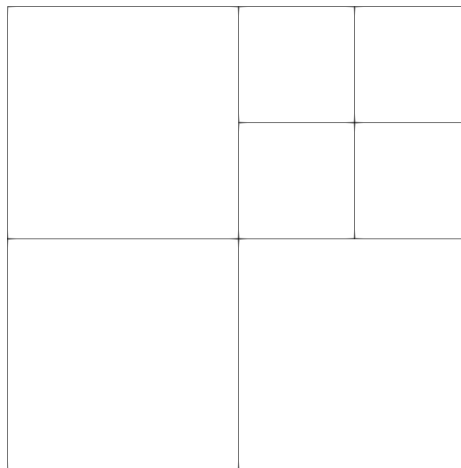


Figura 2.4: Malha gerada no tutorial 4

Nessa parte apenas iteramos sobre todas as células e imprimimos no terminal o id e a localização do centro da célula. Na primeira linha, declaramos o iterador. Nos argumentos do `for`, a função `higcit_isfinished()` será sempre a condição de parada de um iterador de células e a função `higcit_nextcell()` funciona como incremento. A criação do iterador (e a definição de sobre quais células ele irá iterar) acontece com a função `higcit_create_all_leaves()`.

Existe uma série de funções `higcit_create_...()` e outras formas de criar iteradores (relacionadas com domínios, serão comentadas nos próximos tutoriais). O leitor é encorajado a procurar sobre essas funções no header file da biblioteca dos iteradores.

Dentro do loop, usamos o método `higcit_getcell()` para obter a célula que o iterador está apontando atualmente. Obtendo a célula, podemos encontrar as coordenadas do centro dela usando `hig_get_center()` e o id usando `hig_get_cid()`. Cada célula tem um id único com relação à todas as HiG-Trees criadas (isso precisa ser checado).

Por fim, desalocamos o iterador usando `higcit_destroy()`.

```
higfit_facetiterator *fit;
cit = higcit_create_all_leaves(root);

printf("\ncenters of the facets, \tdirection of the facet\n");

int interest_dimensions[2] = {0,1};
for(fit = higfit_create_allfacets(cit, interest_dimensions);
!higfit_isfinished(fit); higfit_nextfacet(fit)) {
    hig_facet *f = higfit_getfacet(fit);
    Point fcenter;
    hig_get_facet_center(f, fcenter);

    int dim = hig_get_facet_dim(f);
    int dir = hig_get_facet_dir(f);
    int id = hig_get_fid(f);

    printf("id: %d \t(x= %.2f, y= %.2f), \tdim: %d \tdir = %d\n",
id, fcenter[0], fcenter[1], dim, dir);
}
```

```
higfit_destroy(fit);
```

Os iteradores de face funcionam de forma análoga aos iteradores de células, assim como as funções dentro do loop. A maior diferença é na criação. Podemos criar um iterador com as faces das células apontadas por um iterador de células, como no código acima. Nesse caso, ao usar `higfit_destroy()`, destruímos o iterador de células também.

Faces possuem uma dimensão e uma direção. A dimensão se refere à direção do vetor normal à face. Por exemplo, se `dim=0`, a reta normal à face está na direção x . A direção nos dá o sentido (não foi eu quem deu os nomes para isso!) do vetor normal à face, considerando que ele sempre aponta para fora da célula. Pode assumir os valores de 0 ou 1, com 0 representando o sentido negativo e 1 o sentido positivo.

Ao rodar o código, vemos que algumas faces possuem a mesma coordenada. Isso pode causar alguns problemas, dependendo do uso. Posteriormente, será mostrada uma função que resolve esse problema. Além disso, cada face possui um id diferente, mesmo que possuam a mesma coordenada.

2.5 Tutorial 5 - Domínios e condições de contorno

Um domínio na biblioteca *higtrees* contém os elementos que possibilitam a solução de uma EDO ou EDP. Nele, armazenamos as HiG-Trees que compõe a malha (pode ser mais de uma), as condições de contorno e um *mapper*, usado para definir ids globais, muito importante quando estamos rodando códigos em paralelo ou para usar o solve, como será feito na seção 2.6. Além disso, ele armazena um interpolador, de forma que não seja preciso gastar esforços com sua alocação, destruição e componentes internos. O código desse tutorial, com o nome *tutorial05-06-2d.c*, é compartilhado com o tutorial 6 Pois, para resolver uma equação de Poisson, precisamos de um domínio, assim como o que será a seguir:

```
#include "domain.h"
#include "mapper.h"
```

Incluimos as bibliotecas que são novidade desse tutorial.

```
POINT_ASSIGN_SCALAR(lo, -1.0);
POINT_ASSIGN_SCALAR(hi, 1.0);
hig_cell *root1 = hig_create_root(lo, hi);
hig_refine_uniform(root1, nc);
FILE *f1 = fopen("VTKS/tut05sd1.vtk", "w");
higio_print_in_vtk2d(f1, root1);

POINT_ASSIGN_INTS(nc, 5, 5);
POINT_ASSIGN_REALS(lo, 1.0, -1.0);
POINT_ASSIGN_REALS(hi, 2.0, 1.0);
hig_cell *root2 = hig_create_root(lo, hi);
hig_refine_uniform(root2, nc);
FILE *f2 = fopen("VTKS/tut05sd2.vtk", "w");
higio_print_in_vtk2d(f2, root2);
```

Criamos duas HiG-Trees que comporão o nosso domínio. Esse domínio é um retângulo com a proporção das células diferente para cada HiG-Tree.

```
sim_boundary *sb[4];
for(int i = 0; i<4; i++){
    mp_mapper *bcm = mp_create();
    switch (i) {
        case 0: // left boundary (x = -1.0)
            POINT_ASSIGN_INTS(nc, 1, 10);
```

```

        POINT_ASSIGN_REALS(lo, -1.0 - EPSDELTA, -1.0);
        POINT_ASSIGN_REALS(hi, -1.0 + EPSDELTA, 1.0);
        break;
    case 1: // top boundary (y = 1.0)
        POINT_ASSIGN_INTS(nc, 15, 1);
        POINT_ASSIGN_REALS(lo, -1.0, 1.0 - EPSDELTA);
        POINT_ASSIGN_REALS(hi, 2.0, 1.0 + EPSDELTA);
        break;
    case 2: // right boundary (x = 2.0)
        POINT_ASSIGN_INTS(nc, 1, 5);
        POINT_ASSIGN_REALS(lo, 2.0 - EPSDELTA, -1.0);
        POINT_ASSIGN_REALS(hi, 2.0 + EPSDELTA, 1.0);
        break;
    case 3: // bottom boundary (y = -1.0)
        POINT_ASSIGN_INTS(nc, 15, 1);
        POINT_ASSIGN_REALS(lo, -1.0, -1.0 - EPSDELTA);
        POINT_ASSIGN_REALS(hi, 2.0, -1.0 + EPSDELTA);
        break;
}
hig_cell *sbc = hig_create_root(lo, hi);
hig_refine_uniform(sbc, nc);

it = higcit_create_all_leaves(sbc);
mp_assign_from_celliterator(bcm, it, 0);
higcit_destroy(it);

it = higcit_create_all_leaves(sbc);
sb[i] = sb_create(sbc, DIRICHLET, bcm);
for(; !higcit_isfinished(it); higcit_nextcell(it)) {
    hig_cell *bcell = higcit_getcell(it);
    Point bccenter;
    hig_get_center(bcell, bccenter);
    int bcgid = mp_lookup(bcm, hig_get_cid(bcell));
    sb_set_value(sb[i], bcgid, u(bccenter[0], bccenter[1]));
}

char buffer[100];
snprintf(buffer, sizeof(buffer),
"VTKS/tut05-sb%d.vtk", i);
FILE *fd = fopen(buffer, "w");
higio_print_in_vtk2d(fd, sbc);
}

```

Nesse trecho do código criamos 4 condições de contorno, uma para cada lado do retângulo que forma o domínio. Elas são armazenadas na estrutura `sim.boundary`, que consiste basicamente de uma HiG-Tree com valores associados. Como é uma HiG-Tree, precisamos criar uma célula raiz para cada condição de contorno (BC). Note que BCs são objetos com dimensão $DIM - 1$ (nesse caso, 1) e, para imitar esse comportamento, as HiG-Trees criadas têm sua "largura" definida como a menor possível (No código, `EPSDELTA`). Dessa forma, ao refinarmos a raiz das BCs, obtemos linhas com diferentes pontos que possuirão valores (no caso 3D, teríamos planos).

Tomando como exemplo a BC da esquerda, refinamos-a com 10 células no eixo y e 1 célula no eixo x . A célula raiz possui ponto inferior logo à esquerda do ponto inferior do domínio e ponto superior logo

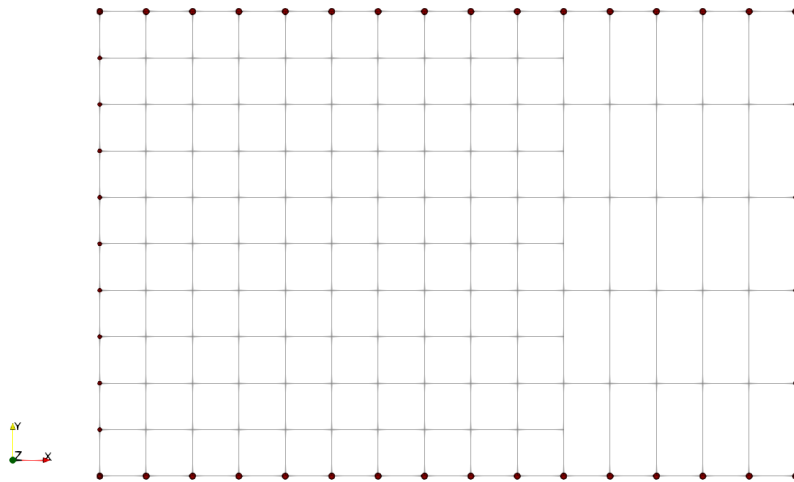


Figura 2.5: Malha gerada no tutorial 5

à direita da face mais à esquerda do domínio.

Note que, cada BC precisa de um *mapper*, Essa estrutura é uma hash-table que leva os ids locais de HiG-Tree em ids para um domínio ou BC inteiro. Dizemos que esse último id é um id global. Quando utilizamos `mp_assign_from_celliterator()`, estamos definindo esse mapeamento. Para as BCs, isso é de extrema importância, visto que os valores associados a cada célula são acessado por esse id global. Para acessar esse id global, usamos `mp_lookup()`, com o argumento sendo o id local da célula.

Nesse caso, estamos usando condições de contorno de Dirichlet. para definir os valores em cada célula, usamos `sb_set_value()`. O id do argumento é o id global.

No fim, obtemos o domínio da figura 2.5. A malha à esquerda é a `root1`, a da direita é a `root2`. Os pontos em vermelho na fronteira são as faces das células das condições de contorno.

```
mp_mapper *mp = mp_create();
sim_domain *sd = sd_create(mp);
sd_add_higtrees(sd, root1);
sd_add_higtrees(sd, root2);

for(int i =0; i < 4; i++) {
    sd_add_boundary(sd, sb[i]); /*Adds all boundaries*/
}
```

Após tudo isso, criamos de fato o domínio, representado pela estrutura `sim_domain`. Adicionamos então as malhas e as condições de contorno ao domínio.

```
it = higcit_create_all_leaves(root1);
int size = mp_assign_from_celliterator(mp, it, 0);
higcit_destroy(it);

it = higcit_create_all_leaves(root2);
size = mp_assign_from_celliterator(mp, it, size);
higcit_destroy(it);
```

Por fim, fazemos o assignment para o *mapper* das células do domínio. a variável `size` guarda a quantidade de células no domínio. Isso será importante para o próximo tutorial.

2.6 Tutorial 6 - Poisson 2D

Usando o domínio criado no tutorial anterior, resolvemos a equação de Poisson

$$\nabla^2 u(x, y) = g(x, y), (x, y) \in \Omega$$

Com condições de contorno

$$u(x, y) = f(x, y), (x, y) \in \partial\Omega$$

Para a discretização do laplaciano, será empregada a discretização por diferenças finitas centradas:

$$\frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{\Delta x^2} + \frac{U_{i,j+1} - 2U_{i,j} + U_{i,j-1}}{\Delta y^2} = g(x_i, y_j)$$

Usaremos o método da solução manufaturada para verificar erros, dessa forma, tomaremos f como uma restrição da solução analítica u para a fronteira.

Duas estruturas serão essenciais na resolução desse problema: o *stencil* e o *solver*. O *stencil* é a estrutura que guarda o peso de cada vizinho relacionado a uma célula específica, pesos esses usados para definir o sistema linear a ser resolvido. O *solver* é responsável por montar o sistema linear e resolvê-lo.

```
#include "solver.h"
```

Precisamos apenas acrescentar essa biblioteca. o *stencil* está implementado em `domain.h`.

```
real u(real x, real y);  
real g(real x, real y);
```

Essas funções são como descritas acima. No arquivo do tutorial, há uma série de casos para esse problema.

```
sim_stencil *stn = stn_create();  
solver *slv = slv_create(SOLVER_ANY, 0, size);  
sd_set_interpolator_order(sd, 3);  
slv_set_maxnonzeros(slv, 300);
```

Criamos o *solver* e o *stencil*. O *solver* possui implementações com diversas engines. A engine padrão utiliza a biblioteca PETSc. Definimos aqui a ordem do polinômio interpolador para 3 (1.3.1). Por fim, definimos a quantidade máxima de entradas não nulas por linha na matriz do sistema. Para cada ponto não central do stencil, usa-se no máximo 60 pontos para interpolação. Como usamos 4 pontos não centrais, 300 entradas não nulas são suficientes.

```
for(it = sd_get_domain_celliterator(sd);  
!higcit_isfinished(it); higcit_nextcell(it)) {  
    hig_cell *c = higcit_getcell(it);  
    Point ccenter;  
    Point cdelta;  
    hig_get_center(c, ccenter);  
    real cx = ccenter[0];  
    real cy = ccenter[1];  
    hig_get_delta(c, cdelta);  
    real dx = cdelta[0];  
    real dy = cdelta[1];  
    dy *= 1.0;  
  
    /* Defines the points for a 5-point stencil*/
```

```

Point center = {cx      , cy      };
Point left   = {cx - dx, cy      };
Point right  = {cx + dx, cy      };
Point top    = {cx,      cy + dy};
Point bottom = {cx,      cy - dy};

stn_reset(stn);
stn_set_rhs(stn, g(cx, cy)); // right side

sd_get_stencil(sd, center, center,
(-2.0/(dx*dx)-2.0/(dy*dy)), stn);
sd_get_stencil(sd, center, left,    (1.0/(dx*dx)), stn);
sd_get_stencil(sd, center, right,   (1.0/(dx*dx)), stn);
sd_get_stencil(sd, center, top,     (1.0/(dy*dy)), stn);
sd_get_stencil(sd, center, bottom,  (1.0/(dy*dy)), stn);

int *ids = stn_get_ids(stn);
real *vals = stn_get_vals(stn);
int numelems = stn_get_numelems(stn);

int cgid = mp_lookup(mp, hig_get_cid(c));
slv_set_Ai(slv, cgid, numelems, ids, vals);
slv_set_bi(slv, cgid, stn_get_rhs(stn));
}
higcit_destroy(it);

```

Iteramos em todas as folhas de todas as árvores do domínio (criamos o iterador usando `sd_get_domain_celliterator()`). Para cada ponto apontado pelo iterador, pegamos os vizinhos. Os pesos dos vizinhos são calculados usando a função `sd_get_stencil()`. Então extraímos esses pesos usando `stn_get_ids()` e `stn_get_values()`. Esses valores são colocados na linha pertencente ao ponto central usando `slv_set_Ai()`. Note que o stencil também calcula o lado direito do sistema, se adaptando as condições de contorno, o que possibilita-nos definir o rhs usando `slv_set_bi()`. Para cada iteração, resetamos o stencil.

```

slv_assemble(slv);
slv_solve(slv);

```

Usamos essas funções para montar o sistema definitivamente e resolvê-lo. Podemos então acessar os resultados calculados usando `slv_get_xi()` ou `slv_get_x()`.

2.7 Tutorial 7 - Equação de Helmholtz vetorial nas faces

Neste tutorial, resolvemos uma equação de Helmholtz vetorial nas faces do domínio. O problema pode ser escrito através de duas equações desacopladas:

$$-\nabla^2 u(x, y) + k^2 u(x, y) = f_1(x, y)$$

$$-\nabla^2 v(x, y) + k^2 v(x, y) = f_2(x, y)$$

A variável u representa a solução ao longo do eixo x e é armazenada nas faces de dimensão 0, enquanto a variável v é a solução no eixo y , armazenada nas faces de dimensão 1. O código adotado encontra-se no arquivo *tutorial07-2d.c*.

Como estamos lidando com variáveis nas faces, precisamos criar domínios de faces (`sim_facet_domain`). O trecho a seguir ilustra a criação de dois domínios (um para cada eixo):

```

sim_facet_domain *sfd[2];
for(int dim = 0; dim < 2; dim++) {
    sfd[dim] = sfd_create(NULL, dim);
    sfd_add_higtrees(sfd[dim], root);
    set_dirichlet_boundary_conditions(sfd[dim]->cdom, lo, hi,
                                     nc, dim == 0? u : v);
    sfd_set_interpolator_order(sfd[dim], 3);
    sfd_adjust_facet_ids(sfd[dim]);
}

```

A função `sfd.adjust_facet_ids()` é crucial aqui, pois lida com faces que possuem a mesma coordenada de centro, ajustando seus identificadores.

Em seguida, o código atribui identificadores globais iterando pelas faces de interesse.

```

int dim_of_interest[2] = {1-dim, dim};
higcit_celliterator *cit = higcit_create_all_leaves(
    sfd_get_higtrees(sfd[dim], 0));
higfit_facetiterator *fit = higfit_create_all_facets(cit,
    dim_of_interest);

size = mp_assign_from_facetiterator(
    sfd_get_domain_mapper(sfd[dim]), fit, start);
start = size;
higfit_destroy(fit);
}

```

O vetor `dim_of_interest` define qual dimensão estamos iterando. Atribuímos os IDs de u primeiro e os de v em seguida (através da variável `start`). Isso possibilita montarmos apenas um sistema linear para resolução. Isso não é vantajoso nesse caso, mas é necessário no caso em que as variáveis são acopladas.

A matriz é montada e resolvida de forma análoga ao exemplo anterior.

2.8 Tutorial 8 - Poisson paralelo 2D

Nesta seção, avançamos para a resolução de equações diferenciais em paralelo, particionando o domínio entre múltiplos processadores com o uso da biblioteca MPI. O código base está no arquivo *tutorial08-2d.c*.

Primeiramente, inicializamos as variáveis do MPI para resgatar o identificador local (`myrank`) e o número total de processos (`ntasks`):

```

MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
MPI_Comm_size(MPI_COMM_WORLD, &ntasks);

```

Para dividir as árvores que compõem a malha, introduzimos o grafo de partição (`partition_graph`) e o balanceador de carga (`load_balancer`):

```

partition_graph *pg = pg_create(MPI_COMM_WORLD);

sim_domain *sd = sd_create(NULL);
sd_create_boundary(root, sd, DIRICHLET);
set_dirichlet_boundary_conditions(sd);

load_balancer *lb = lb_create(MPI_COMM_WORLD, 1);
if (myrank == 0) {

```

```

        lb_add_input_tree(lb, root, true, 0);
    } else {
        hig_destroy(root);
    }
    lb_calc_partition(lb, pg);

```

O `loadbalancer` é o responsável por efetivamente calcular a partição das HiG-Trees através da função `lb_calc_partition()`. Após isso, resgatamos a parcela do domínio que pertence ao processador atual e adicionamos ao domínio base `sd`.

O passo final e mais importante da paralelização é criar o domínio particionado e sincronizar os IDs globais através da rede:

```

psd = psd_create(sd, pg);
psd_synced_mapper(psd);
int localdomainsize = psd_get_local_domain_size(psd);

```

A estrutura `psim_domain` encapsula o domínio para uso paralelo. A chamada de `psd_synced_mapper()` faz automaticamente todo o trabalho pesado de sincronização de IDs no mapeador, o qual, caso contrário, precisaria ser feito manualmente. Com a matriz e o vetor local definidos, o *solver* resolve o sistema linear distribuído e salva um arquivo `.vtk` para o domínio de cada processador, facilitando a visualização dos resultados parciais.

Note que neste tutorial, estamos lendo a malha a partir de um arquivo `.amr` (Adaptive Mesh Refinement). Na seção 2.10, será explicado de forma detalhada como esses arquivos são usados.

2.9 Tutorial 9 – Equação de Advecção e Métodos Numéricos

Nesta etapa vamos explorar um problema dependente do tempo, resolvendo a equação de advecção linear:

$$u_t + v_1 u_x + v_2 u_y = 0$$

assumindo velocidades positivas $v_1, v_2 > 0$ e uma condição inicial $u(x, y, 0) = f(x, y)$. Para lidar com a evolução temporal, utilizamos a estrutura `distributed_property` (propriedade distribuída), que armazena os valores da simulação em um dado instante e permite consultar esses valores passados para calcular o próximo passo de tempo.

```

distributed_property *dpu_last = psd_create_property(psd);
distributed_property *dpu_now = psd_create_property(psd);

```

Para partir para um próximo passo temporal, antes temos que usar a função `dp_sync(dpu_last)` para que, se estivermos rodando o código em paralelo, atualizamos as fringes. Foram usados dois métodos distintos para lidar com as derivadas espaciais desta equação: Lax-Friedrichs e Upwind, respectivamente nos arquivos `tutorial09-lax-2d.c` e `tutorial09-upwind-2d.c`. Para Lax, é necessário definir uma BC de outflow nas bordas direitas e superior, dado que o método usa pontos downwind.

```

for(int t = 0; t < total_iterations; t++) {
    dp_sync(dpu_last);
    for(it = sd_get_domain_celliterator(localdomain);
        !higcit_isfinished(it); higcit_nextcell(it)) {
        hig_cell *c = higcit_getcell(it);
        Point ccenter, cdelta;
        hig_get_center(c, ccenter);
        hig_get_delta(c, cdelta);
        real dx = cdelta[0];
        real dy = cdelta[1];
    }
}

```

```

Point bottom = {ccenter[0], ccenter[1] - dy};
Point left = {ccenter[0] - dx, ccenter[1]};

int cgid = mp_lookup(m, hig_get_cid(c));

real uleft = sd_dp_interpolate(localdomain, dpu_last,
                               ccenter, left, stn);
real ubottom = sd_dp_interpolate(localdomain, dpu_last,
                                 ccenter, bottom, stn);
real ucenter = dp_get_value(dpu_last, cgid);
real next_u = ucenter - ((v1*dt)/dx)*(ucenter-uleft)
              - ((v2*dt)/dy)*(ucenter-ubottom);

dp_set_value(dpu_now, cgid, next_u);
}
dp_copy_values(dpu_last, dpu_now);
time += dt;
higcit_destroy(it);
}

```

Utilizando o código com o upwind implementado para comentar, vemos cinco funções principais utilizadas para trabalhar com as *distributed properties*. A primeira, já comentada é a `dp_sync()`. A função `sd_dp_interpolate()` calcula a interpolação da *distributed property* no ponto definido, utilizando os métodos de interpolação já discutidos (existe a variante `sfd_dp_interpolate()` para domínios de faces). `dp_get_value()` simplesmente retorna o valor em um determinado id e `dp_set_value()` define um valor para o id. Por fim, `dp_copy_values` copia os valores de uma *distributed property* para outra.

2.10 Arquivos .amr

Arquivos `.amr` armazenam a estrutura de uma malha, ou seja, consegue representar um domínio ou até uma condição de contorno. Quanto a sua estrutura, ela possui os seguintes blocos:

Na primeira linha, temos as coordenadas dos pontos $high = (x_2, y_2)$ e $low = (x_1, y_1)$ do retângulo (paralelepípedo) que compõe a célula raiz.

```

x1  x2  y1  y2
-1.0 1.0 -1.0 1.0

```

No caso tridimensional, colocamos `x1 x2 y1 y2 z1 z2`.

Na segunda linha, colocamos o número de níveis de Refinamento

```

2

```

Nas próximas linhas, descrevemos os níveis de refinamento. Cada nível possui um número de *patches* e o tamanho de cada célula após o refinamento. Cada *patch*, por sua vez, define a célula inicial a se aplicar o refinamento e a quantidade de células, a partir do ponto inicial, a serem refinadas.

```

primeiro nivel
delta_x delta_y  num_patches
0.05      0.05      1
patch 1

```

```

x_ini y_ini  qnt_x qnt_y
1      1      20   20

segundo nivel
delta_x delta_y  num_patches
0.025   0.025    2
patch 1
1 1 40 40
patch 2
41 41 40 40

```

A posição da célula inicial e a quantidade de células para se aplicar o patch são baseadas na malha uniforme que seria feita ao dividir toda a raiz pelo delta desse nível. No exemplo acima, no primeiro nível temos $(dx, dy) = (0.05, 0.05)$. Como o domínio tem tamanho 2 para as duas direções, temos que a quantidade de células é $(\frac{2}{0.05}, \frac{2}{0.05}) = (40, 40)$. Na linha do primeiro patch, 1 1 40 40 estamos falando para iniciar na primeira célula e andar 40 células (com espaçamento de 0.05) para ambas as direções, percorrendo todo o domínio. Note que a quantidade de células inclui a célula inicial.

No segundo nível, temos no total $(80, 80)$ células. Fazemos dois patches que são um quadrado de $1/4$ do domínio.

Por fim, obtemos o seguinte `.amr` para essa malha:

```

-1.0 1.0 -1.0 1.0
2
0.05 0.05 1
1 1 40 40
0.025 0.025 2
1 1 40 40
41 41 40 40

```

Que gera a malha da figura 2.6.

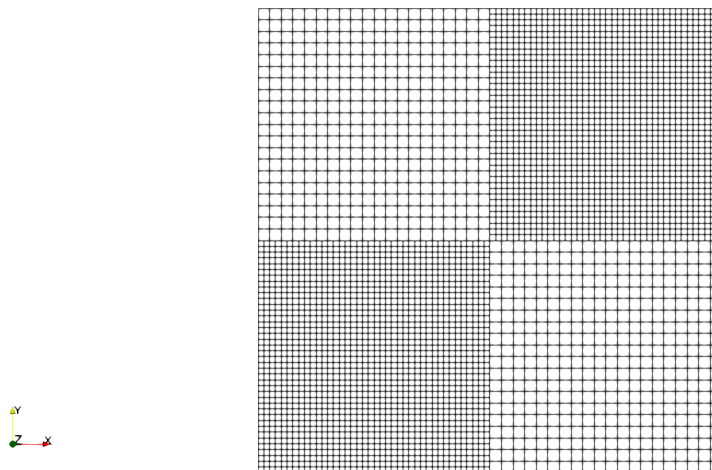


Figura 2.6: Malha gerada pelo `.amr`

Para usar arquivos `.amr` na biblioteca `higtree`, o módulo `higtree-io` proporciona duas maneiras para a leitura desses arquivos. A primeira utiliza a função `higio_read_from_amrDIMd()` e retorna imediatamente a `hig_cell` correspondente. A segunda utiliza a função `higio_read_amr_info()`, que lê o arquivo e o transforma na estrutura `higio_amr_info`, que pode ser utilizada em outras funções do pacote de I/O da `higtree`.

Documentação do programador

Grafo de dependências da biblioteca higtree:

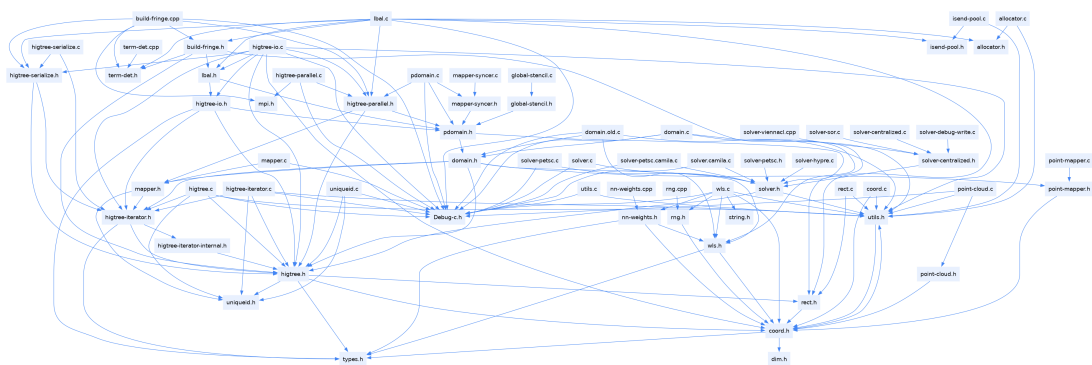


Figura 3.1: Grafo geral

Referências Bibliográficas

- [1] Alexandre Joel Chorin. Numerical solution of the Navier–Stokes equations. *Mathematics of Computation*, 22(104):745–762, 1968.
- [2] T. M. Shih, C. H. Tan, and B. C. Hwang. Effects of grid staggering on numerical schemes. *International Journal for Numerical Methods in Fluids*, (2):193–212, 1989.